

FORMAL TECHNICAL ASSESSMENT REPORT

ORCHESTRA-OS

Final Product, Evidence, Strength, Weakness, and Professional Capability Assessment

Translation of the professor's simulation research into a real Linux scheduling prototype

Prepared by:	S.W. ZAW
Report date:	2 September 2026
Repository revision:	94664aeb001d8b3552245aedfd78254fbb5f13b8
Validated platform:	ThinkPad T14 Gen 1; Kali Linux; kernel 7.0.12
Document status:	Final evidence-synthesis report
Product status:	Research-grade release candidate

Claim boundary. This report distinguishes simulation, userspace validation, kernel prototyping, and bounded real-machine validation. It does not present ORCHESTRA-OS as deployment-ready or as a universal replacement for Linux CFS/EEVDF.

Document Control

Field	Value
Purpose	State what was built and discovered, identify strong and weak areas, and map demonstrated abilities to suitable professional work.
Primary research basis	Ganesan and Vaithiyanathan, 6 July 2026 simulation paper on predictive, hierarchical, signal-coordinated process scheduling.
Primary product evidence	Frozen real-machine and validation archives through 1 September 2026.
Report type	Technical assessment report; not a new research paper.
Evidence status	38 PASS, 2 preserved FAIL, 7 BLOCKED in the principal 47-item checklist; 81/81 bounded runtime assertions passed.
Current-worktree warning	The repository was dirty at inspection. Results are attributed to recorded revisions, artifacts, hosts, and protocols.
Change control	Documentation only; no source, test, BPF, scheduler parameter, or machine configuration was changed.

Intended readers: project supervisor, academic assessors, technical reviewers, prospective employers, and engineering collaborators.

Contents

Document Control	i
1 Executive Summary	1
2 Research Basis and Product Translation	2
3 Validation Method and Evidence Quality	5
4 Product Results	7
5 Engineering Discoveries	9
6 Strength and Weakness Assessment	10
7 Professional and Work-Fit Assessment	12
8 Recommendations and Roadmap	14
9 Final Conclusion	15
A Evidence Reference Register	16
B Reproducibility Note	18

1. Executive Summary

1.1 Overall judgment

ORCHESTRA-OS converted substantial parts of the professor’s pre-kernel simulation architecture into an opt-in Linux `sched_ext` prototype. The evidence covers a userspace control plane, bridge, versioned BPF-map contracts, exact task admission, five bounded actions, requested-versus-effective telemetry, invalid or stale input rejection, target-matched attach, and clean unload.

ORCHESTRA-OS is stronger as an opt-in scheduler control and observability prototype than as a general replacement for Linux CFS/EEVDF.

1.2 Decision dashboard

Table 1.1: Headline evidence and interpretation.

Area	Result	Meaning
Runtime actions	81/81 assertions passed	All five canonical actions were observed in bounded probes with effectiveness, fallback, or release evidence.
Checklist	38 PASS; 2 FAIL; 7 BLOCKED	Negative and unavailable evidence was retained rather than hidden.
Foreground protection	20/20 wins; 74.2405% mean paired reduction	Strong service-policy result for one controlled photo/archive protocol.
Background service cost	97.6851% fewer mean CPU ticks	The foreground gain came from deliberate service reallocation, not free acceleration.
Pure CPU work	1.58x–2.05x CFS time	Clear negative result for general CPU performance under the tested artifact and protocol.
Mixed CPU/I/O work	0.86x–0.94x CFS time	Promising but exploratory result from three repetitions per cell.
Signal publication	12/12 runs; 12,000 publications; 27,161 reads	Strong userspace evidence; not kernel cryptographic proof.
Bounded soak	600 s each for CPU, I/O, and mixed	Ownership and health checks passed; memory ownership remained blocked.

1.3 Professional capability conclusion

The strongest demonstrated abilities are research-to-code translation, Linux/eBPF prototyping, systems validation, causal failure diagnosis, observability and lifecycle design, and reproducible technical communication. The best-supported job families are Systems Research Engineer, Research Software Engineer, Linux/eBPF Engineer, Performance or Benchmarking Engineer, Systems Validation Engineer, and Observability/Reliability Engineer.

2. Research Basis and Product Translation

2.1 Discoveries inherited from the professor’s study

The source paper is a simulation study, not a hardware deployment report. Its numerical claims remain **SIMULATED**.

Table 2.1: Simulation discoveries and implementation consequences.

Discovery	Simulation evidence	Product consequence
Temporal blind spot	S1/S2/S3 allowed synchronized mass switching to look perfect	Add S4 so instability and herd behavior reduce Q.
Aggregation artifact	A raw product mechanically shrank when a healthy factor was added	Use a geometric mean for cross-version interpretation.
Wrong controller target	A PID actuator saturated while the deficit was in other submetrics	Map actuators to metrics they can causally affect.
Learning/control coupling	Inner learning and outer control changed one another	Enforce explicit two-timescale separation.
Reward contradiction	Reward encouraged RUN where the directive required MIGRATE; correction raised S2 from 0.57 to 0.79	Jointly audit reward and directive definitions.
State-bin mismatch	A bucket contained states with different correct actions; realignment raised S2 to 0.90	Align representations to decision boundaries.
Predictor failure	Online adaptive Kalman MSE was about seven times worse than simple fixed alternatives	Prefer offline calibration when noise is not identifiable.
Missing causal credit	Agents lacked feedback for their group contribution	Blend difference reward with a clear local objective.
Simulation gate	Seven failures were cheaper to find before kernel work	Preserve simulation as an architectural validation step.

2.2 Development path

The inspected HEAD contains 85 commits from the first userspace prototype on 5 August 2026 to the runtime-hardening revision on 25 August 2026.

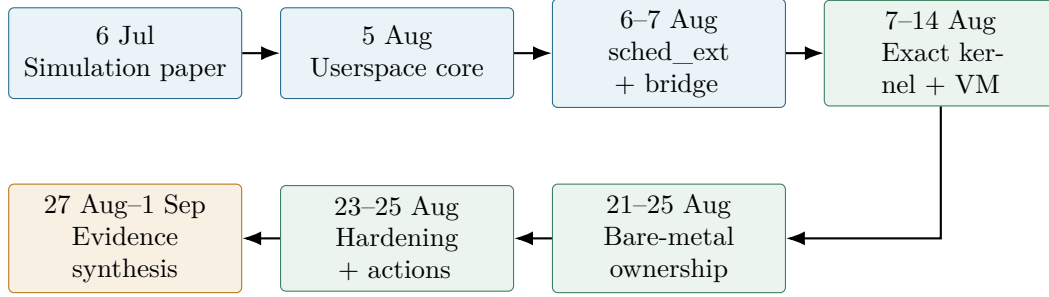


Figure 2.1: Development and validation progression. Color distinguishes design and implementation, kernel validation, and evidence synthesis; it does not indicate deployment readiness.

2.3 Implemented architecture

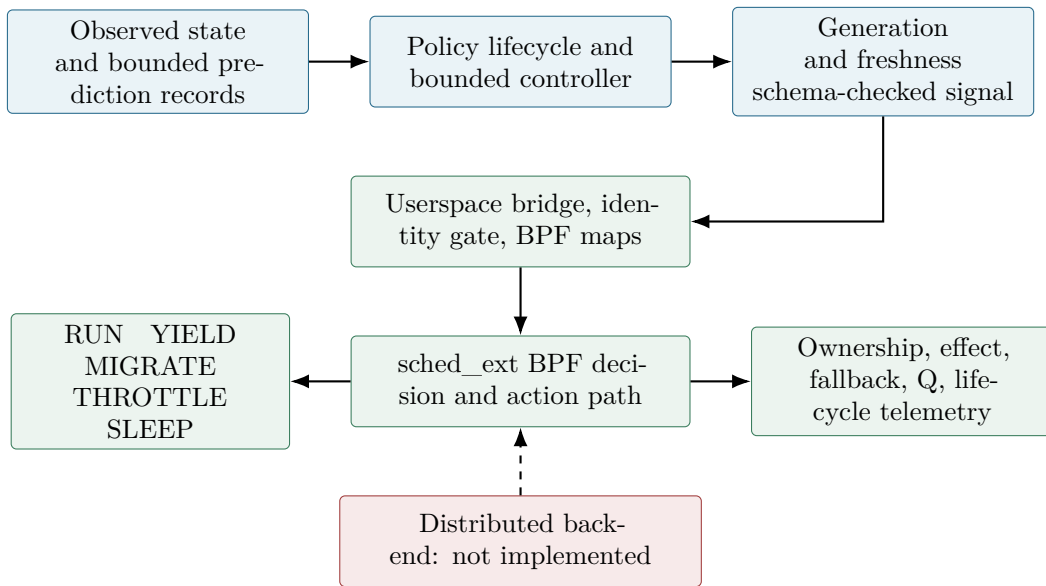


Figure 2.2: Implemented single-host path and explicit non-implemented boundary.

Table 2.2: Research-to-product implementation matrix.

Component	Strongest evidence	Boundary
Scheduler, bridge, loader	Target verifier, attach, ownership, unload	One target/API family; not general kernel portability.
Five action backends	Bounded real-machine action matrix	Not all workload, fairness, topology, or timing conditions.
Signal bus	Userspace integrity and kernel generation/freshness gates	No HMAC verifier in BPF.
Prediction	Confidence, expiry, generation, observed fallback contract	Hardware prediction accuracy and benefit untested.
S1/S2/S3/S4/Q	Corrected contract and limited live windows	No broad population statistical validation.
Controller	Bounded contract and limited transitions	Full causal actuator and rollback matrix open.
RT boundary	Admission refusal and bounded FIFO/RR coexistence	No hard-RT guarantee.
NUMA/distributed	One-node observation; no distributed backend	Cross-node NUMA blocked; distributed tier absent.

3. Validation Method and Evidence Quality

3.1 Evidence chain

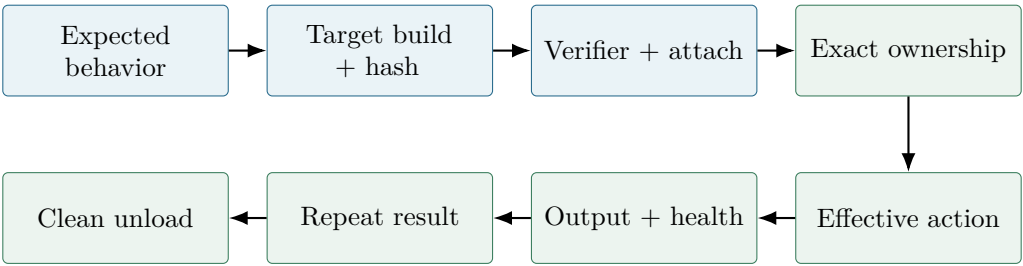


Figure 3.1: Evidence chain required for a bounded real-machine claim.

3.2 Checklist status

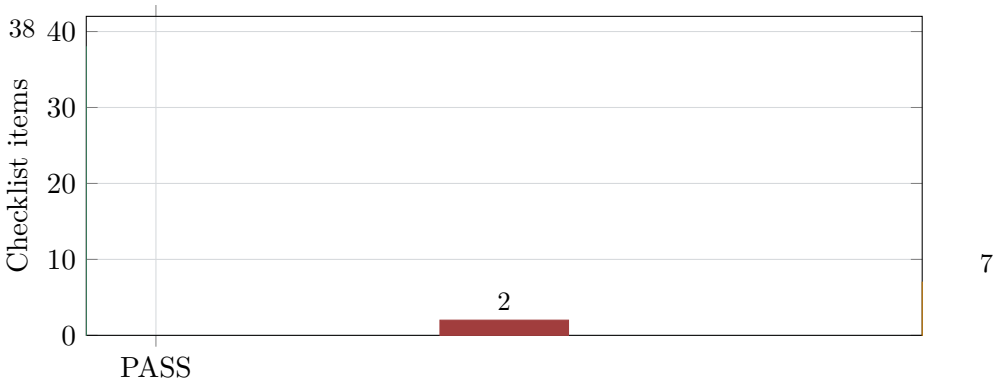


Figure 3.2: Principal 47-item checklist. PASS applies only to the named bounded test and does not upgrade blocked deployment claims.

3.3 Representative action evidence

Table 3.1: Bounded action observations from the 81/81 matrix.

Action	Observed evidence	Boundary
RUN	Dispatch 31; running 34; effective 34	Controlled forward progress.
YIELD	45 specialized dispatches; 44 effective; peer progress	Bounded queue-tail behavior, not universal latency.
MIGRATE	28 accepted; 28 dispatched; 27 effective; requested, dispatched, and actual CPU all 1	Same-node movement only.
Invalid MI-GRATE	44 bad-CPU fallbacks; zero MI-GRATE acceptance or dispatch; effective RUN fallback	Fail-closed for this invalid target.
SLEEP	Acceptance, defer, release, resumed progress	One-shot bounded defer/release evidence.
THROTTLE	Immediate, after-RUN, rollover, re-publication, and two-CPU cases	Broad fairness and bandwidth semantics remain open.

4. Product Results

4.1 General fixed-work performance

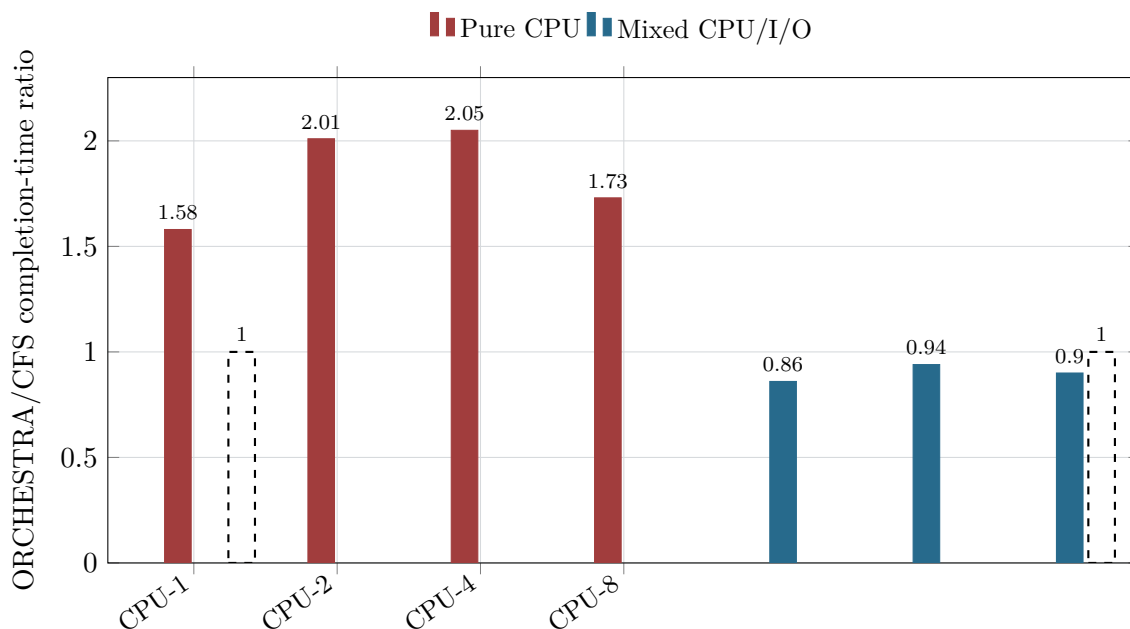


Figure 4.1: Fixed-work results, N=3 per scheduler per cell. Pure CPU is negative; mixed CPU/I/O is directionally positive but exploratory.

Pure CPU is the clearest performance weakness. A complete cause requires matched `perf`, tracepoint, cache, and target-matched `scx_simple` evidence, which was unavailable.

4.2 Strongest result: foreground protection

The held-out protocol ran four foreground ImageMagick resizes with eight bzip2 background workers for 20 counterbalanced pairs. Output, ownership, effective THROTTLE, and clean-unload gates passed.



Figure 4.2: Foreground means. ORCHESTRA won 20/20 pairs; mean paired reduction was 74.2405%.



Figure 4.3: Service cost. Mean background CPU ticks fell 97.6851%.

The graphs must be read together. ORCHESTRA protected foreground work by restricting background service. Background completion, total makespan, and a matched CFS cgroup or CPU-quota policy were not measured. This is service-policy evidence, not proof of better total-system efficiency.

4.3 Integrity, stress, and real-time boundary

- Userspace publication passed 12/12 runs, 12,000 publications, and 27,161 verified reads. BPF contains no HMAC verifier.
- Ownership-gated CPU, I/O, and mixed phases ran 600 seconds each with no recorded errors or warnings and clean unload. Memory remained blocked.
- FIFO/RR coexistence preserved protected tasks outside the adaptive path. DEADLINE contention was blocked by host EPERM; hard RT is not proven.
- A log/archive result showed 5/5 wins and a 60.4678% mean paired reduction, but 96–97 °C measurements make it thermally confounded.

5. Engineering Discoveries

1. **Correctness is a chain, not a success message.** Build, verifier, attach, ownership, request, acceptance, dispatch, effect, fallback, and unload are distinct claims.
2. **Exact kernel matching is a release constraint.** Kernel API, BTF, headers, toolchain, and loader identity must be recorded per target.
3. **Observability is a product feature.** Requested-to-effective telemetry made stale generation, migration fallback, deferred release, controller state, and cleanup diagnosable.
4. **Service policy differs from speed.** Background CPU service changed the interpretation of the foreground improvement.
5. **Exact TID admission does not cover every application.** Some nominally single-threaded tools expose multiple active TIDs.
6. **Transitions are high-risk.** A reproduced RUN-to-THROTTLE bottleneck showed the importance of generation and accounting synchronization.
7. **Negative results improve credibility.** CPU slowdown, predictor failure, thermal confounding, one-node NUMA, and the absent distributed backend correctly narrowed the claim.

6. Strength and Weakness Assessment

Ratings summarize evidence in this project, not personal intelligence or future potential.

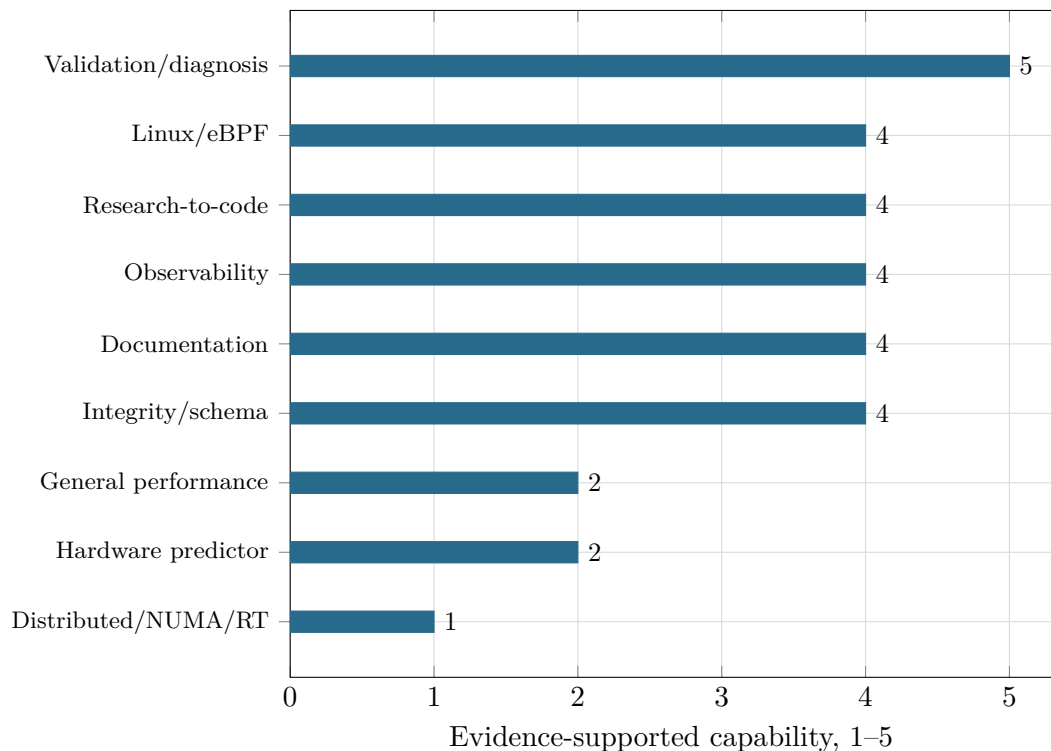


Figure 6.1: Capability assessment from direct repository evidence.

6.1 Strongest areas

1. Evidence-led systems validation and diagnosis;
2. Linux/eBPF and `sched_ext` prototyping;
3. research-to-code translation;
4. observability, identity, fallback, and lifecycle design;
5. reproducibility and formal technical communication; and
6. controlled foreground service policy within one validated protocol.

6.2 Weak or incomplete areas

- Pure-CPU performance and context-switch cost;
- hardware predictor accuracy and measurable scheduling benefit;
- full controller-actuator causality and rollback;
- whole-application ownership for multithreaded software;

- kernel cryptographic authentication and key lifecycle; and
- hard RT, cross-node NUMA, hotplug, distributed scheduling, and production readiness.

7. Professional and Work-Fit Assessment

7.1 Best-fit job families

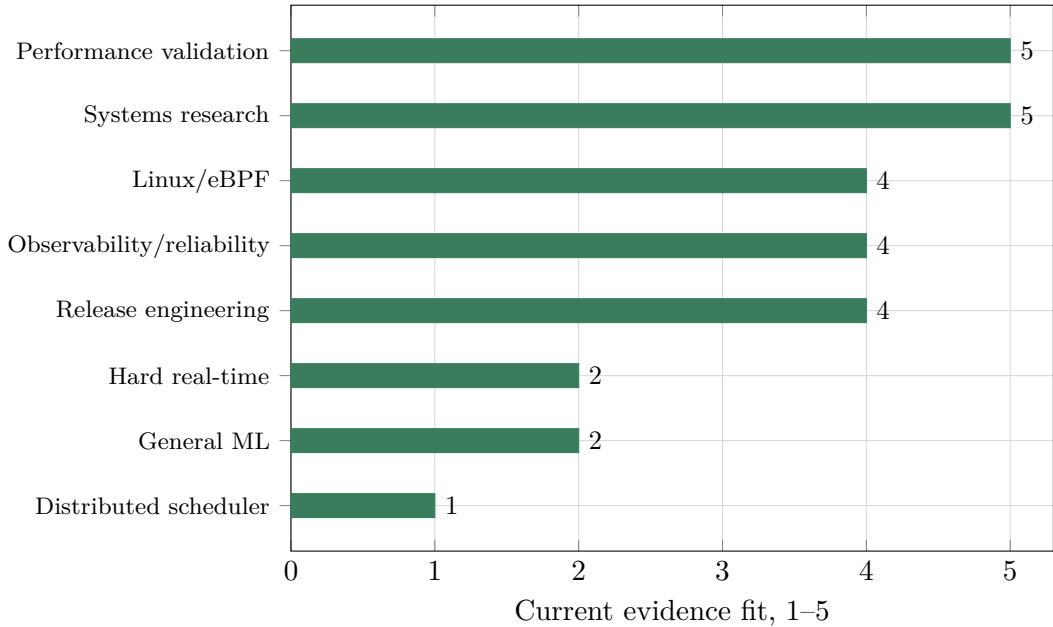


Figure 7.1: Job-family fit based only on evidence demonstrated by ORCHESTRA-OS.

Table 7.1: Professional fit and honest boundary.

Job family	Fit	Evidence and boundary
Systems Research / Research Software Engineer	Excellent	Paper-to-prototype translation, controlled experiments, and negative-result diagnosis. Describe it as bounded, not production-ready.
Performance / Benchmarking Engineer	Excellent	Baselines, ownership, counterbalancing, repetitions, confounder detection, and service-cost analysis. Do not claim universal speed.
Linux Systems / eBPF Engineer	Strong	BPF maps, target matching, loader lifecycle, <code>sched_ext</code> , identity, and verifier-aware testing. Larger production kernel coverage remains open.
Systems Validation Engineer	Excellent	Action, integrity, stress, RT boundary, fault/recovery, and checklist evidence with preserved failures.
Observability / Reliability Engineer	Strong	Requested-to-effective telemetry, lifecycle, fallbacks, health, and causal diagnosis. Production uptime is not represented.
Edge Resource-Management Engineer	Strong	Opt-in service differentiation and mixed CPU/I/O fit constrained worker pools. Field confirmation is required.

Job family	Fit	Evidence and boundary
Native Build / Release Engineer	Good	Kernel manifests, ABI/schema gates, hashes, packaging, and reproducibility. Cross-platform coverage remains open.
Hard RT / Distributed / NUMA-HPC / Crypto Assurance	Not yet supported	Relevant implementation or validation is blocked, absent, or too narrow.

7.2 Best kinds of work

The evidence suggests the team is best suited to projects where a research architecture must become a bounded prototype; behavior crosses userspace and kernel boundaries; instrumentation is as important as implementation; failures must be reproduced and explained; and identity, safety, fallback, lifecycle, and reproducibility matter.

Suitable work includes eBPF policy engines, kernel observability tools, resource governors, edge workload controllers, systems test platforms, fault/recovery harnesses, native-software release pipelines, and research prototypes.

7.3 Defensible portfolio statement

Built and validated a research-grade, opt-in Linux `sched_ext` control and observability prototype that translated predictive scheduling research into five bounded kernel actions with exact ownership, fallback, and lifecycle evidence.

Claims to avoid: ORCHESTRA is faster than Linux; deployment ready; kernel signal authentication is proven; hard real-time is proven; distributed scheduling is implemented; or prediction improves hardware scheduling.

8. Recommendations and Roadmap

8.1 Highest-value next experiment

The next decisive test is a matched service-policy comparison, not another broad demonstration.

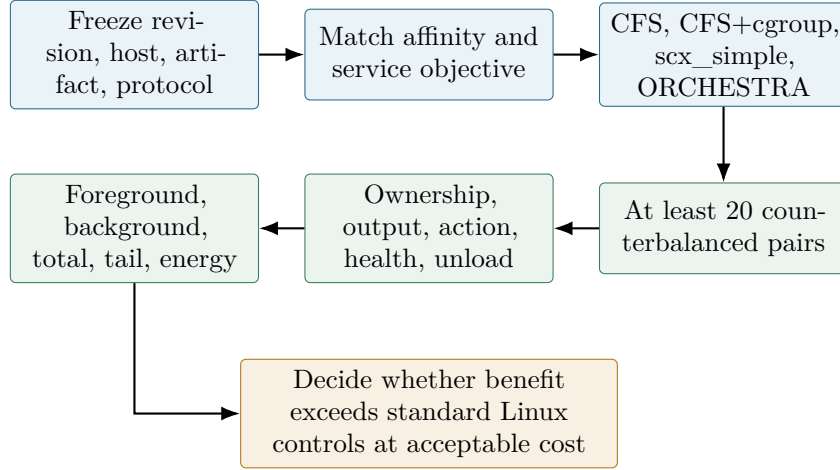


Figure 8.1: Recommended evidence path for the next product decision.

Table 8.1: Priority gap-closure plan.

P	Gap	Required evidence	Claim unlocked
1	Matched policy baseline	CFS+cgroup and ORCHESTRA with identical service objectives and at least 20 pairs	Value beyond standard Linux controls.
2	CPU overhead	perf , tracepoints, callbacks, migrations, cache misses, and matched scx_simple	Causal overhead diagnosis.
3	Whole-application ownership	Thread-group or cgroup admission and per-thread telemetry	Defensible application control.
4	Predictor/controller	Dynamic regimes, error, confidence, actuator response, saturation, rollback	Hardware predictive/control evidence.
5	Kernel trust boundary	Threat model, key lifecycle, provenance, fuzzing, independent review	Narrow secure-system claim.
6	Platform maturity	Multi-node NUMA, hotplug, x86/arm matrix, long soak, upgrade and rollback	Broader release readiness.

9. Final Conclusion

ORCHESTRA-OS has achieved more than a paper reproduction and less than a production scheduler. It has a coherent research-to-code mapping, a target-specific working `sched_ext` prototype, exact ownership and action evidence, strong observability, bounded stress and recovery evidence, and a reproducible foreground service-control result.

It has not demonstrated a general CFS/EEVDF performance advantage, hardware predictor benefit, kernel-side cryptographic verification, broad fairness or tail latency, hard-RT safety, cross-node NUMA, distributed scheduling, or deployment readiness.

ORCHESTRA-OS is a research-grade, opt-in Linux scheduler control and observability platform with bounded real-machine validation of ownership, five actions, fallback, lifecycle, and foreground service protection. General performance and production-readiness claims remain open.

The strongest professional story is that the team took an ambitious operating-systems research architecture, identified where its assumptions failed, translated its strongest ideas into a real Linux/eBPF prototype, proved what the kernel did, preserved negative results, and defined the evidence required for the next product claim.

A. Evidence Reference Register

Bibliography

- [1] G. Ganesan and P. Vaithiyathan, *ORCHESTRA-OS: Simulation-Driven Architectural Discovery and Design Principles for Predictive, Hierarchical, Signal-Coordinated Process Scheduling*, 6 July 2026. Repository PDF SHA-256: 9af8f45412e2f7d8be89f108965372caa2c420f917c7b612c3b4a86ab626561d.
- [2] ORCHESTRA-OS, *Work Package Description*, WORK_PACKAGE_DESCRIPTION.md.
- [3] ORCHESTRA-OS, *Research-to-Code Mapping*, docs/research/RESEARCH_TO_CODE.md.
- [4] ORCHESTRA-OS, *Final Product Status*, FINAL_PRODUCT_STATUS.md.
- [5] S.W. ZAW, *ORCHESTRA-OS versus Linux CFS: Validation Report Package*, 1 September 2026, including provenance, audit, evidence calculations, implementation matrix, checklist, action matrix, fixed-work data, paired foreground statistics, long-run results, and checksums.
- [6] S.W. ZAW, *ORCHESTRA-OS Real-World Field-Fit Report*, 27 August 2026.
- [7] S.W. ZAW, *ORCHESTRA-OS Strongest Part and Real-World Comparison Guide*, 27 August 2026.

B. Reproducibility Note

No new scheduler run was performed to create this report. Numerical claims were taken from the frozen validation package whose audit records successful checksum verification and recalculation. Because the current worktree contains staged and untracked changes, a future campaign must freeze a new revision, capture repository and machine state, rebuild the exact target artifact, and repeat the relevant gates.